PlacementIQ Enterprise: AI-Powered Placement and Career Readiness System

Shaganti Vivek Goud
Dept. of Computer Science and Engineering
Anurag University
Hyderabad, India
23eg105f59@anurag.edu.in

Puram Preethi
Dept. of Computer Science and Engineering
Anurag University
Hyderabad, India
23eg105f52@anurag.edu.in

Bobbala Akshaya Reddy
Dept. of Computer Science and Engineering
Anurag University
Hyderabad, India
23eg105f03@anurag.edu.in

Bandari Laxmisri
Dept. of Computer Science and Engineering
Anurag University
Hyderabad, India
23eg105f09@anurag.edu.in

Dr. Kamalakar Ramineni
Assistant Professor
Dept. of Computer Science and Engineering
Anurag University
Hyderabad, India
kamalakar.cse@anurag.edu.in

***Abstract*—Most engineering students get their first job through campus placements, but few of them know which visiting companies they actually qualify for, or what is holding them back. Placement cells, for their part, still filter students by CGPA cut-offs passed around in spreadsheets and WhatsApp groups. We built PlacementIQ Enterprise to change this. It is a placement platform shared by several colleges, and it compares a student's academics, skills, projects, internships, resume and coding activity with the requirements of 66 recruiters and of the drives a college posts. For each company the student gets one of four labels (Eligible, Nearly Eligible, Can Become Eligible, Not Eligible) together with the criteria they missed. The platform also estimates placement readiness, a likely salary band and interview readiness, and a What-If simulator lets a student try out an improvement before working on it. A language model helps with resume review, job-description matching and a career assistant that answers from the student's own data; its use is limited by a monthly quota. What students, placement cells, recruiters and administrators may see is decided by the database itself, through row-level security. We tested the engine on 10,000 synthetic students (660,000 decisions). Scoring one student against every company took 0.18 ms on average, every non-eligible decision came with a reason, and 594,000 monotonicity checks found no violation. Compared with plain cut-off screening, 80.3% of the pairs that pass the cut-off still have a major skill gap. On the live database, all 19 access-control tests passed.**

***Keywords*—Campus placement, eligibility classification, explainable artificial intelligence, skill-gap analysis, large language models, role-based access control**

I. Introduction

In India, most engineering graduates find their first job through campus recruitment, so placement numbers matter to students, to colleges and to accreditation bodies alike. Several studies use machine learning to predict from academic and demographic records whether a student will be placed [1]-[3]. Their results agree on one point: marks explain only part of the outcome. Skills, projects and internships count too.

Work on career guidance has moved forward in parallel. Skill-assessment tools map students to job roles [4], recommenders suggest what to study next [5], e-recruitment systems match candidates to openings at scale [6], and large language models (LLMs) are being tried as tutors and explainers in education [7].

In a college placement office, though, these ideas seldom come together. A predictor that outputs “placed” or “not placed” gives a student nothing to act on, and explainability is still a known weak spot of the models that do get deployed [8]. Meanwhile, company criteria travel through portals, spreadsheets, e-mail and chat groups, and students are shortlisted on CGPA alone, even though the companies go on to test coding, design and aptitude.

PlacementIQ Enterprise addresses this gap with a single platform in which each college onboards its own students, posts its own drives and monitors every year, branch and section, while each student sees, for every company, an explainable eligibility category, the exact gaps to close, and simulated outcomes of planned improvements.

Contributions of this Paper

- A multi-college placement platform with database-enforced roles for students, placement cells, company recruiters and platform administrators, roster-only student access and per-plan AI usage quotas.
- An explainable multi-criteria eligibility engine that classifies each student against 66 recruiters into four actionable categories and lists every unmet criterion, together with transparent readiness, expected-salary and interview-readiness models and a What-If simulator.
- Resume review with six ATS-style checks, job-description matching and a career assistant, all built on an LLM that is given the student's own records as context.
- An evaluation that anyone can rerun: 660,000 synthetic decisions measured for latency, explanation coverage and monotonicity and compared with cut-off screening, plus 19 access-control scenarios tested on the live database.

Section II covers related work. Section III explains how the system is designed and built, Section IV reports what we

measured, and Section V closes with conclusions and next steps.

A. Motivation

- In one season a college may host dozens of drives, each with its own CGPA, backlog, branch, batch and skill rules. Students read every notice by hand, and many only learn they missed a requirement after being rejected.
- The information sits in many places at once (portals, spreadsheets, e-mail, WhatsApp). A CGPA cut-off also tells nobody whether the student can get through the coding, design or aptitude rounds.
- If eligibility were always up to date and explained, placement officers would stop re-filtering the same lists, students could start preparing months ahead, and the college could see readiness by year, branch and section.
- We started the project while preparing for our own placements at Anurag University. The department also wanted real numbers on how ready each batch was.

B. Problem Statement

We are given a student profile x and a set of companies C. The profile holds CGPA, backlogs, Class X and XII scores, eight skill levels, projects, internships, a resume and coding-platform statistics; each company c has requirements r_c. For every pair (x, c) the system must pick one of four ordered categories and list the requirements that are not met. It must also estimate placement readiness, expected salary and interview readiness, and show each result only to users who are allowed to see it.

- Classify a student against all companies in under 1 ms of computation, so that the dashboard and What-If simulator update instantly.
- Attach at least one concrete, unmet criterion or skill gap to 100% of non-eligible decisions, and guarantee that an improvement in any input never lowers a match score or category (monotonicity).
- Isolate data between colleges and roles so that 100% of defined access-control scenarios pass on the production database.

II. Literature Survey (Papers from IEEE, ACM, and Springer)

Placement and employability prediction. Basha et al. [1] explored institutional placement data and then trained supervised classifiers on it. Krishnaiah and Kadegowda [2] mixed several machine-learning methods to forecast whether engineering undergraduates get jobs, and Kunwar et al. [3] went further with an attention-based CNN-BiLSTM that also suggests skills to learn. These models predict reasonably well. Still, each gives one answer per student, not one per company, and a deep network like [3] is hard to explain to the student it scores. A systematic review [8] reached the same conclusion: explainable prediction is rare in practice.

Career guidance and learning recommendation. C3-IoC [4] measures how close a student's skills are to different IT roles and draws the result as a network, so the student can explore options. Bin et al. [5] survey how e-learning systems model learners and pick what to recommend. Kasneci et al. [7] weigh what LLMs can and cannot safely do in classrooms, and Bhatnagar [9] uses an LLM to work out what users are asking a career chatbot. All of this is general career advice. None of it looks at the actual criteria of the companies that visit one particular campus.

Person-job fit, e-recruitment and resume matching. Qin et al. [10] train a neural network on past hiring records to judge how well an applicant's abilities fit a job. Yang et al. [11] treat hiring as a choice made from both sides, by the applicant and by the employer. Mashayekhi et al. [6] organise e-recruitment recommenders around their open challenges, among them sparse data, explainability and fairness. For text, BERT [13] and Sentence-BERT [14] turn resumes and job descriptions into vectors that can be compared and ranked [12], and the SkillSpan dataset [15] labels hard and soft skills inside job ads. Almost all of this is built for recruiters and needs large labelled hiring data. A student gets no feedback from it.

Evidence from developer platforms. RepoSkillMiner [16] reads a developer's GitHub repositories with NLP to name their skills, and Sun et al. [17] estimate expertise with a graph neural network over who follows and contributes to what. Both show that real coding activity says more than a skills list a student types in, which is why our platform pulls data from coding sites.

A. Comparative Summary of Related Work

Table I compares representative studies by year, method, data and main limitation. Most of them predict a single placement outcome, recommend careers or learning in general, or rank candidates for recruiters; none of them tells a student, for each visiting company, whether they are eligible, why not, and what to improve.

TABLE I
COMPARISON OF EXISTING APPROACHES

Ref.	Year	Method	Dataset	Limitation
[1]	2023	EDA + supervised ML classifiers	Institutional placement records	Placed or not only; no per-company criteria
[2]	2022	Hybrid machine learning	Engineering graduate records	Outcome prediction; no improvement plan
[3]	2025	CNN-BiLSTM with attention	Student academic and skill data	Black-box model; weak explanation
[4]	2023	Skill similarity + job-role network	IT job roles, student skills	Career exploration, not campus drives
[5]	2024	Review of learning recommenders	Studies 2013-2023	Learning-centric; no recruiter criteria
[6]	2024	Survey of e-recruitment recommenders	Published systems	Explainability and fairness open
[8]	2021	Review of explainable prediction	Studies 2015-2020	Few deployed explainable models
[9]	2025	LLM intent mining for chatbots	Career-guidance queries	Not grounded in student records
[10]	2018	Ability-aware neural person-job fit	Recruitment histories	Needs large hiring data; recruiter-side
[12]	2023	NLP and BERT resume ranking	Resumes, job descriptions	Ranks candidates; no student feedback
[15]	2022	Skill extraction corpus	English job postings	Extraction only; no matching to students
[17]	2024	GNN on GitHub social network	GitHub developers	Expertise only; no placement context

B. Research Gap

The studies above do one of three things: predict a single placement outcome from marks, suggest careers or courses with no link to the drives a college actually receives, or rank applicants for recruiters. We did not find a system that tells each student, company by company, whether they qualify, what is missing and what would change if they improved. Nor did we find one that does this for several colleges at once while keeping their data apart and giving placement cells and recruiters their own views. PlacementIQ Enterprise is our attempt to cover that gap.

III. PROPOSED METHODOLOGY / IMPLEMENTATION

A. System Overview

The architecture is shown in Fig. 1. All four kinds of users open the same React single-page application, which sends each role to its own set of pages. Eligibility, predictions and the What-If simulator are computed in the browser from the student's own data, so nothing waits for a server. The data is stored in Supabase (PostgreSQL). There, row-level security policies, the role model of [18] and a few private helper functions decide which rows a request can read or write [19]. Secret keys never reach the browser: they stay in serverless edge functions, which handle resume analysis, job-description matching, the career assistant (using gpt-oss-120b [20]), syncing of coding profiles, roster sign-up and invitations.

Colleges and their placement-cell logins are created by a platform administrator. Each placement cell then uploads its roster with e-mail, name, roll number, branch, graduation year and section, posts its drives and adds the recruiters of visiting companies. Only e-mail addresses registered by these roles can hold an account; this is enforced by a trigger on the authentication table.

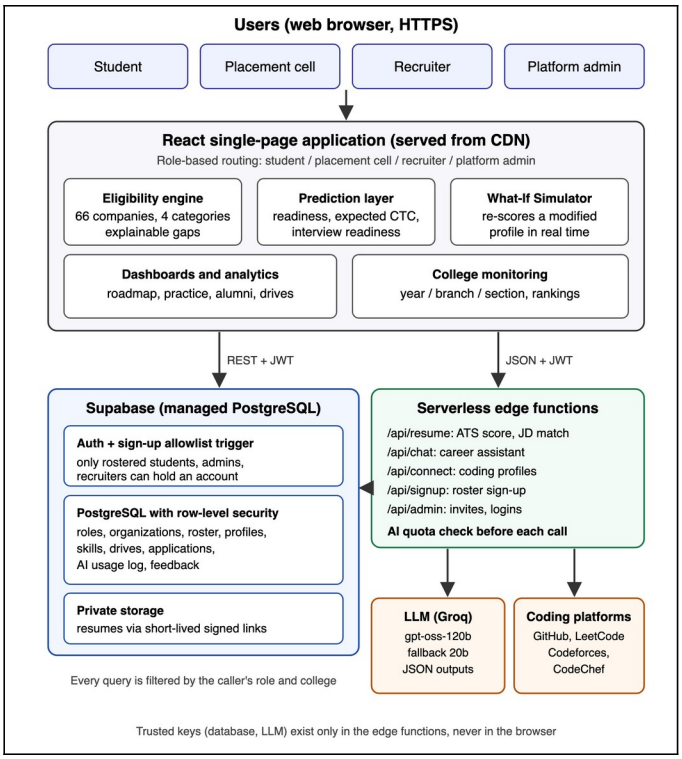


Fig. 1. Proposed system architecture of PlacementIQ Enterprise.

B. Dataset / Data Collection

PlacementIQ does not train a model on a fixed dataset; it scores live records. Student data comes from mandatory onboarding and the profile: CGPA, active backlogs, Class X and XII percentages, levels (0-100) for eight skills (data structures and algorithms, system design, frontend, backend, databases, machine learning, cloud and DevOps, aptitude), projects, internships, certifications and achievements. Resumes are parsed in the browser (PDF with pdf.js, DOCX with mammoth); coding statistics are fetched from the public GitHub, LeetCode, Codeforces and CodeChef interfaces.

Company data covers 66 recruiters grouped into ten categories (for example top product, quant, AI, fintech, startup, IT services). Each category defines minimum CGPA, maximum backlogs, Class X and XII minimums, internships, projects and eight skill levels, with a small deterministic per-company variation; drives posted by a placement cell or recruiter carry their own explicit criteria and job description. Skill levels are normalized to 0-100, CGPA to 0-10 and percentages to 0-100.

To evaluate the engine we generated 10,000 synthetic students with a seeded random generator. CGPA ranges from 5.5 to 9.8 and 82% of the students have no backlogs. A hidden ability value makes skill levels correlate, as they do in real batches, and each student gets 0-5 projects and 0-2 internships. Because the seed is fixed, every number in this paper can be reproduced.

C. Algorithm / Model Design

Take a student x and a company c with requirements r_c. The engine starts with an academic score A. It divides the student's CGPA g, Class X score X and Class XII score Y by the company minimums (starred), caps each ratio at 1, adds a backlog term b (1 within the limit, 0.4 otherwise) and averages the four:

$$A = 25\,[\hat{g} + X + \hat{Y} + b],\quad \hat{g} = \min(1, g/g^{*}) \tag{1}$$

For skills, we add up the shortfall on every skill k, where r_k is the required level and s_k the student's level. Several small gaps therefore hurt, much as they would across several interview rounds:

$$S = \max(0,\ 100 - 1.2\,\Sigma_k \max(0,\ r_k - s_k)) \tag{2}$$

Experience E and projects P are also capped ratios against what the company asks for, and original GitHub repositories count as projects. The match score is then:

$$M = 0.15\,A + 0.60\,S + 0.10\,E + 0.15\,P \tag{3}$$

The category is decided in a fixed order. When an academic minimum is not met, the pair becomes Can Become Eligible if the student is within 0.5 CGPA and 5 percentage points of each minimum and $M \geq 55$; if not, it is Not Eligible. When all minimums are met, the pair is Eligible if $M \geq 88$, the largest gap $\Delta\max \leq 5$ and the internship requirement holds; Nearly Eligible if $M \geq 75$, $\Delta\max \leq 15$ and the total gap ≤ 25; Can Become Eligible if $M \geq 55$; and Not Eligible otherwise. The criteria and the gaps (largest first) are saved with each decision and shown to the student. Placement readiness is the mean match over the target categories T (top product, product, startup and fintech):

$$R = (1/|T|)\,\Sigma_{c \in T}\, M_c \tag{4}$$

Expected salary weights each company's average package by an offer likelihood $w_c = \omega(\text{category}_c) \cdot M_c / 100$, with ω = 0.55, 0.30, 0.10 and 0.01 for the four categories; the 25th and 75th weighted percentiles give the likely band:

$$\widehat{CTC} = \Sigma_c\, w_c\, CTC_c \,/\, \Sigma_c\, w_c \tag{5}$$

Interview readiness uses six inputs: coding skill D, aptitude T, system design Z, the average mock-interview score K (logged out of 10 and scaled to 100), resume score Q, and experience V, which gives 20 points per project and 25 per internship up to 100:

$$I = 0.3D + 0.15T + 0.15Z + 0.2K + 0.1Q + 0.1V \tag{6}$$

To rank students, placement cells see a composite score. Besides readiness, it uses profile completeness U and a CGPA term $G = \min(100, 10g)$:

$$\text{Score} = 0.35R + 0.2Q + 0.2D + 0.15U + 0.1G \tag{7}$$

The What-If simulator runs the same functions on an edited copy of the profile, where the student can change CGPA, backlogs, skill levels, projects, internships or mock scores. It lists the companies that change category. It also tries raising each skill by 10 and reports which single step helps the most. The LLM features (resume analysis with six ATS checks, job-description matching and the career assistant) receive the structured profile as context and return JSON or text grounded in it. Before any call, one action is deducted from the student's monthly quota.

D. Implementation Details

On the front end we used React 18 and TypeScript with Vite 5 and Tailwind CSS. Charts are drawn with Recharts, and three.js is loaded lazily, only for the 3D visual on the landing page. The back end is Supabase (PostgreSQL with row-level security, authentication and private storage) plus Vercel serverless edge functions. Groq serves the LLM, gpt-oss-120b, and falls back to gpt-oss-20b. Roles (super_admin, org_admin, recruiter) are stored in a user_roles table and checked by SECURITY DEFINER helper functions in a private schema; AI quotas are 15, 40, 80 and 150 actions per student per month for the Trial, Basic, Pro and Enterprise plans. The evaluation scripts ran on Node.js 25 on an Apple Silicon laptop.

E. Evaluation Metrics

Because no labelled placement outcomes are available yet, the system is evaluated on properties that its design requires: (i) latency per student for all 66 companies and all predictions (mean and 95th percentile); (ii) explanation coverage, the share of non-eligible decisions that state at least one unmet criterion or gap; (iii) monotonicity, the share of checks in which raising one skill by 10 or CGPA by 0.5 never lowers a match score or category; (iv) divergence from CGPA/backlog cut-off screening; (v) What-If actionability, the share of students for whom one +10 skill step moves at least one company up a category; and (vi) the pass rate of access-control tests.

IV. Results Analysis and Comparisons

A. Experimental Setup

For the experiments we took the engine and prediction code exactly as deployed and bundled it into an evaluation script (docs/evaluation/evaluate.ts in the project repository). It scored 10,000 synthetic students against 66 companies (660,000 decisions), ran 594,000 monotonicity checks on 1,000 students, and compared every decision with a baseline that accepts a student when CGPA and backlogs meet the company minimum. Access control was tested with 19 scenarios on the production database inside a transaction that was rolled back, so no data was changed.

B. Performance Results

Table II lists the seven measurements E1 to E7 with the result, the number of samples and the outcome of each. Fig. 2 compares CGPA cut-off screening with the proposed multi-criteria classification over the same 660,000 student-company

decisions, and Fig. 3 shows how one decision and its reasons appear to a student in the deployed application.

TABLE II
MEASURED PERFORMANCE OF THE PROPOSED SYSTEM

No.	Metric	Result	Samples	Outcome
E1	Latency per student, mean / p95	0.18 / 0.25 ms	10,000 students	Real time
E2	Non-eligible decisions with a stated reason	100%	629,068 decisions	Pass
E3	Monotonicity violations	0	594,000 checks	Pass
E4	Cut-off accepts with a major skill gap	80.3%	341,912 pairs	Baseline over-admits
E5	Cut-off rejects that can become eligible	10.3%	318,088 pairs	Baseline gives no path
E6	One +10 skill step moves a company up	36.7%	1,000 students	Actionable
E7	Access-control tests passed	19 / 19	19 scenarios	Pass

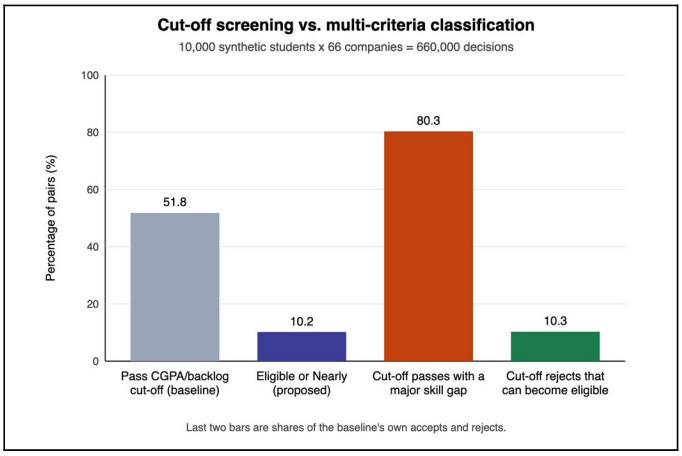


Fig. 2. Cut-off screening compared with the proposed multi-criteria classification (660,000 decisions).

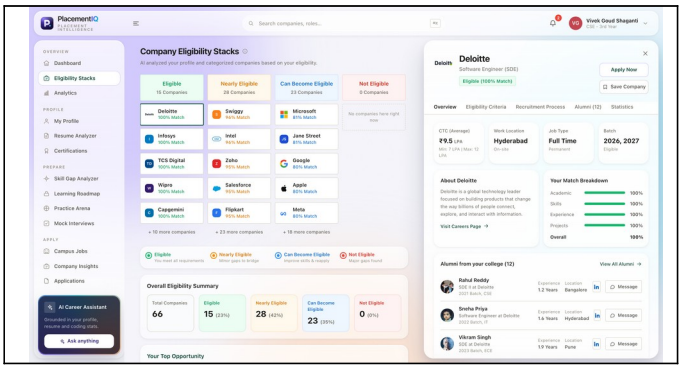


Fig. 3. Company Eligibility Stacks screen with a company panel showing criteria and match breakdown (sample data).

C. Discussion

Table II and Fig. 2 summarize the results. All predictions for one student across 66 companies took 0.18 ms on average (0.25 ms at the 95th percentile), well inside the 1 ms objective, which is why the dashboard and the What-If simulator respond instantly in the browser. Each of the 629,068 non-eligible decisions named at least one unmet criterion or skill gap. Across 594,000 checks, improving an input never lowered a match score or a category, so a student is never penalized for getting better.

The comparison with cut-off screening is the main practical finding. The cut-off accepted 51.8% of all student-company pairs, but 80.3% of those accepted pairs had at least one required skill more than 15 points short or were classified Can Become or Not Eligible, so cut-off lists send many unprepared candidates into drives. Conversely, 10.3% of the pairs rejected by the cut-off were close enough to be Can Become Eligible, a group the cut-off gives no guidance at all. For 36.7% of students a single 10-point improvement in one skill moved at least one company up a category, which the What-If simulator surfaces as the most effective next step. All 19 access-control scenarios passed, covering the sign-up allowlist, roll-number verification, isolation between colleges, recruiter restrictions and the AI quota.

Unlike the predictors in [1]-[3], the proposed engine outputs a decision and a reason per company, and unlike the recruiter-side matchers in [10]-[12] it is designed to guide students. The trade-off is that its thresholds and category weights are expert-defined rather than learned. The evaluation uses a synthetic cohort and checks design properties, not agreement with real placement outcomes, so no accuracy is claimed; company requirement templates are approximations of public hiring patterns; and LLM outputs for resume analysis and the assistant were not benchmarked quantitatively. These limitations define the pilot study planned as future work.

V. CONCLUSION AND FUTURE SCOPE

A. Conclusion

We built PlacementIQ Enterprise so that a student no longer has to guess. Instead of a CGPA cut-off and messages spread over many channels, each student sees, company by company, whether they qualify and why. On 660,000 synthetic student-company pairs the engine needed well under a millisecond per student, gave a reason for every non-eligible result and never punished an improvement. It also showed that 80.3% of the pairs a cut-off would accept still hide a major skill gap. The readiness, salary and interview estimates, the What-If simulator and the resume tools turn those results into a plan the student can follow. Row-level security keeps every college's and recruiter's data separate, and all 19 access-control tests passed.

B. Future Scope

- Pilot the platform with all batches at Anurag University and record who actually gets placed. With those outcomes we can tune the thresholds, and train a learned model (for example gradient boosting explained with SHAP) to compare against the transparent engine.
- Add interview scheduling and offer tracking for recruiters and placement cells, roles for faculty mentors, single sign-on, and analytics that compare colleges.

- Move to AWS once more colleges join, test the system under load, audit the eligibility rules for fairness, and add data-retention and consent settings that meet the Digital Personal Data Protection Act, 2023.

ACKNOWLEDGMENT

We are grateful to our guide, Dr. Kamalakar Ramineni (Assistant Professor, CSE, Anurag University), who guided us through every stage of this project, and to the CSE department for its support and facilities.

REFERENCES

[1] M. S. A. Basha, R. P. K. Raju, P. M. Oveis, and M. Sarim, "Unraveling campus placement success integrating exploratory insights with predictive machine learning models," in *Proc. 7th Int. Conf. Comput. Syst. Inf. Technol. Sustain. Solutions (CSITSS)*, Bengaluru, India, Nov. 2023, pp. 1-6, doi: 10.1109/CSITSS60515.2023.10334118.

[2] V. Krishnaiah and Y. H. Kadegowda, "Undergraduate engineering students employment prediction using hybrid approach in machine learning," *Int. J. Elect. Comput. Eng.*, vol. 12, no. 3, pp. 2783-2791, Jun. 2022, doi: 10.11591/ijece.v12i3.pp2783-2791.

[3] F. B. Kunwar, A. Ramachandran, E. Manigandan, S. Srivastava, S. Priya, and S. Suhane, "Enhanced student placement prediction and skill recommendation system using convolutional neural network-bidirectional long short-term memory with attention networks," in *Proc. 7th Int. Conf. Intell. Sustain. Syst. (ICISS)*, 2025, pp. 1450-1456, doi: 10.1109/ICISS63372.2025.11076538.

[4] A. José-García et al., "C3-IoC: A career guidance system for assessing student skills using machine learning and network visualisation," *Int. J. Artif. Intell. Educ.*, vol. 33, no. 4, pp. 1094-1121, 2023, doi: 10.1007/s40593-022-00317-y.

[5] Q. Bin, M. F. Zuhairi, and J. Morcos, "A comprehensive study on personalized learning recommendation in e-learning system," *IEEE Access*, vol. 12, pp. 100446-100482, 2024, doi: 10.1109/ACCESS.2024.3428419.

[6] Y. Mashayekhi, N. Li, B. Kang, J. Lijffijt, and T. De Bie, "A challenge-based survey of e-recruitment recommendation systems," *ACM Comput. Surv.*, vol. 56, no. 10, pp. 1-33, Oct. 2024, doi: 10.1145/3659942.

[7] E. Kasneci et al., "ChatGPT for good? On opportunities and challenges of large language models for education," *Learn. Individ. Differ.*, vol. 103, Art. no. 102274, 2023, doi: 10.1016/j.lindif.2023.102274.

[8] R. Alamri and B. Alharbi, "Explainable student performance prediction models: A systematic review," *IEEE Access*, vol. 9, pp. 33132-33143, 2021, doi: 10.1109/ACCESS.2021.3061368.

[9] M. Bhatnagar, "Enhancing career guidance through intent mining with large language models," in *Artificial Intelligence in Education Technologies: New Development and Innovative Practices* (Lecture Notes on Data Engineering and Communications Technologies), T. Schlippe, E. C. K. Cheng, and T. Wang, Eds. Singapore: Springer, 2025, pp. 17-33, doi: 10.1007/978-981-97-9255-9_2.

[10] C. Qin et al., "Enhancing person-job fit for talent recruitment: An ability-aware neural network approach," in *Proc. 41st Int. ACM SIGIR Conf. Res. Develop. Inf. Retrieval*, Ann Arbor, MI, USA, 2018, pp. 25-34, doi: 10.1145/3209978.3210025.

[11] C. Yang, Y. Hou, Y. Song, T. Zhang, J.-R. Wen, and W. X. Zhao, "Modeling two-way selection preference for person-job fit," in *Proc. 16th ACM Conf. Recommender Syst. (RecSys)*, Seattle, WA, USA, 2022, pp. 102-112, doi: 10.1145/3523227.3546752.

[12] S. Tanberk, S. S. Helli, E. Kesim, and S. N. Cavsak, "Resume matching framework via ranking and sorting using NLP and deep learning," in *Proc. 8th Int. Conf. Comput. Sci. Eng. (UBMK)*, Burdur, Türkiye, Sep. 2023, pp. 453-458, doi: 10.1109/UBMK59864.2023.10286605.

[13] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, Minneapolis, MN, USA, 2019, pp. 4171-4186, doi: 10.18653/v1/N19-1423.

[14] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. EMNLP-IJCNLP*, Hong Kong, China, 2019, pp. 3980-3990, doi: 10.18653/v1/D19-1410.

[15] M. Zhang, K. Jensen, S. Sonniks, and B. Plank, "SkillSpan: Hard and soft skill extraction from English job postings," in *Proc. NAACL-HLT*, Seattle, WA, USA, 2022, pp. 4962-4984, doi: 10.18653/v1/2022.naacl-main.366.

[16] S. Kourtzanidis, A. Chatzigeorgiou, and A. Ampatzoglou, "RepoSkillMiner: Identifying software expertise from GitHub repositories using natural language processing," in *Proc. 35th IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE)*, 2020, pp. 1353-1357, doi: 10.1145/3324884.3415305.

[17] Y. Sun et al., "Automatically deriving developers' technical expertise from the GitHub social network," in *Proc. 39th IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE)*, 2024, pp. 2462-2463, doi: 10.1145/3691620.3695329.

[18] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, "Role-based access control models," *Computer*, vol. 29, no. 2, pp. 38-47, Feb. 1996, doi: 10.1109/2.485845.

[19] Supabase, "Row Level Security." Supabase Docs. https://supabase.com/docs/guides/database/postgres/row-level-security (accessed Oct. 6, 2026).

[20] OpenAI, "gpt-oss-120b & gpt-oss-20b model card," *arXiv*:2508.10925, Aug. 2025, doi: 10.48550/arXiv.2508.10925.